

Complete Analysis: SatQuery AI - SIH Problem Statement 26167

Executive Overview

This is a highly complex, technically ambitious problem statement from ISRO that requires building a production-grade multimodal AI system. Let me break down everything you need to know to understand, plan, and win this.

PART 1: DEEP UNDERSTANDING OF THE PROBLEM

1.1 What ISRO Actually Wants

text

Simple Terms: Build an AI assistant that can look at satellite images and answer questions about them in plain English - like ChatGPT but for satellite imagery with specialized remote sensing knowledge.

The Core Pain Point They're Solving:

text

Current State:

- Expert needed → understand GIS/satellite data
- Different tools for different tasks (classification, detection, etc.)
- No unified natural language interface
- Non-experts cannot extract insights from satellite imagery

Desired State (SatQuery AI):

- Upload satellite image(s)
- Ask in plain English
- Get intelligent, evidence-backed answers
- Works for farmers, disaster managers, urban planners, etc.

1.2 Input Types You Must Handle

text

INPUT CONFIGURATIONS	
Single Image	One optical/SAR image → VQA (MANDATORY) → Captioning OR Grounding (pick one)
Cross-Modal Pair	Optical + SAR of SAME area → Joint information extraction → Complementary analysis
Bi-temporal Pair	Same area, DIFFERENT times → Change detection (MANDATORY) → Change description → Change VQA

Formats: GeoTIFF/TIFF (primary), PNG/JPEG (benchmarks only)

1.3 Mandatory vs Optional Features

text

MANDATORY (Must Have to Pass):

- ✓ Remote sensing adaptation (fine-tune on BigEarthNet)
- ✓ Single-image VQA
- ✓ Either captioning OR grounding (pick one)
- ✓ Bi-temporal change analysis (description or VQA)
- ✓ Cross-modal optical+SAR analysis
- ✓ Agentic orchestration (auto-select models based on query)

BONUS (Will Win Points):

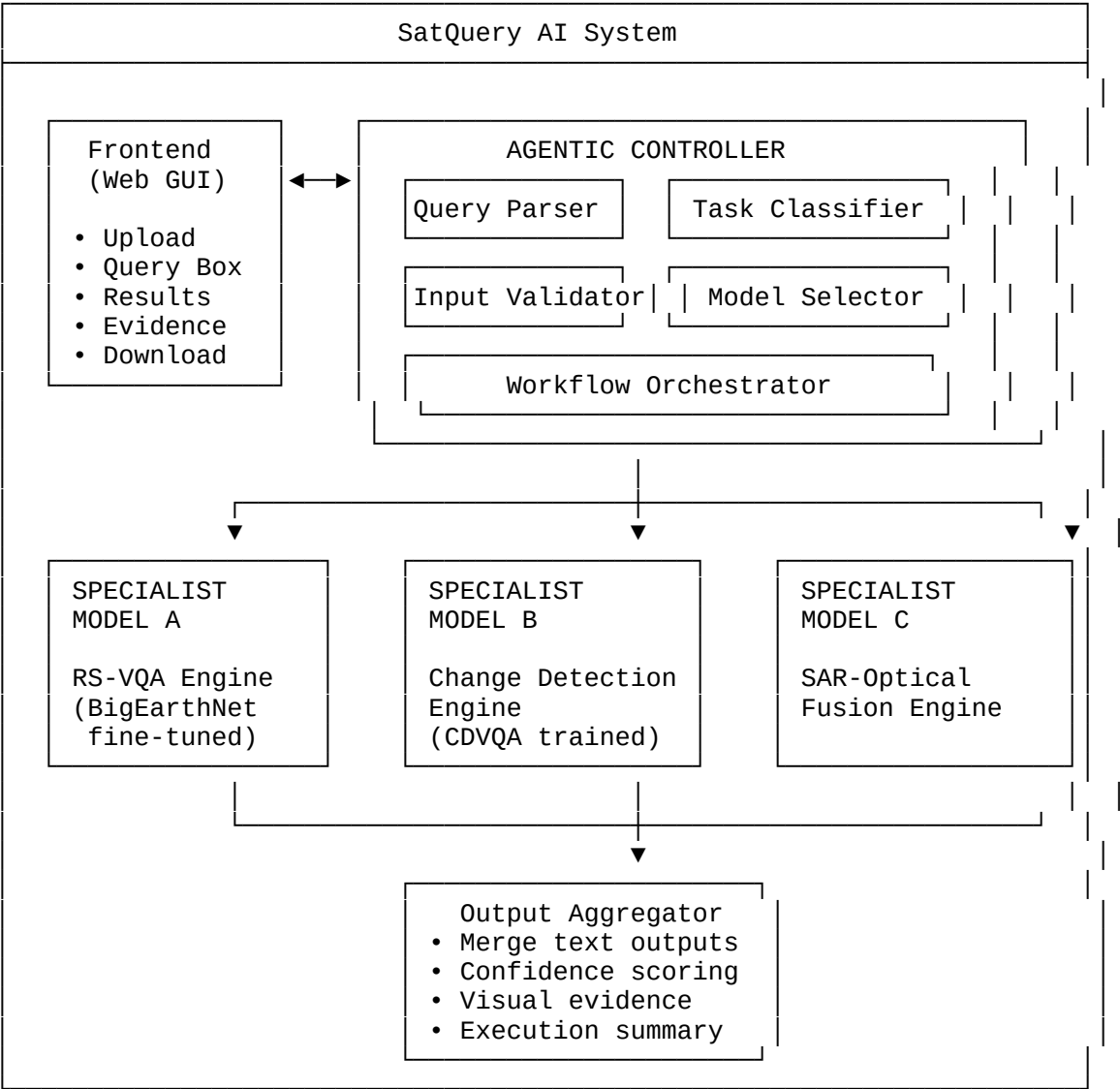
- ★ Both captioning AND grounding
- ★ Spatial change maps with masks
- ★ Confidence scores
- ★ Downloadable reports

★ Auditable execution traces

PART 2: TECHNICAL ARCHITECTURE ANALYSIS

2.1 System Architecture Blueprint

text



2.2 Technology Stack Recommendation

text

LAYER	TECHNOLOGY	WHY
Frontend	React.js + Tailwind Leaflet.js/OpenLayers	Modern, responsive UI Map visualization
Backend API	FastAPI (Python)	Async, ML-friendly
AI Orchestration	LangChain/LangGraph OR Custom Python Agent	Agent framework
LLM Controller	Llama-3.1 (local) OR Mistral-7B	Free, privacy-safe
Vision Encoder	CLIP (RS-adapted) RemoteCLIP	Image understanding Already RS-adapted!
VQA Model	LLaVA-1.6 fine-tuned OR InstructBLIP adapted	Vision-language

Change Detection	ChangeFormer OR BIT (Binary Change)	SOTA change detection
SAR Processing	Custom CNN/ViT	SAR feature extraction
Captioning	GeoChat (fine-tuned)	RS-specific captioning
Grounding	GLIP/Grounding DINO adapted for RS	Text-guided detection
RS Adaptation	BigEarthNet	MANDATORY fine-tuning
Image Processing	GDAL, Rasterio	GeoTIFF handling
Database	PostgreSQL + Redis	Storage + Caching
Deployment	Docker + Nginx	Containerized

2.3 Agentic Controller - The Heart of the System

Python

```
# Conceptual Agent Logic

class SatQueryAgent:

    def process(self, query: str, images: List[Image]) -> Response:

        # STEP 1: Understand the query
        task_type = self.classify_task(query)
        # → "VQA" | "CAPTIONING" | "GROUNDING" |
        #    "CHANGE_DETECTION" | "SAR_OPTICAL_FUSION"

        # STEP 2: Validate inputs
        input_config = self.validate_inputs(images)
        # → Check format, modality, co-registration, timestamps

        # STEP 3: Select models
        models = self.model_registry.select(task_type, input_config)
        # → Returns appropriate specialist models

        # STEP 4: Execute workflow
        raw_outputs = self.execute_workflow(models, images, query)

        # STEP 5: Aggregate & respond
        response = self.aggregate_outputs(raw_outputs)
        # → Text answer + visual evidence + confidence + trace

        return response

    def classify_task(self, query: str) -> TaskType:
        # Use LLM to understand intent
        # "What changed between..." → CHANGE_DETECTION
        # "Describe the..." → CAPTIONING
        # "Highlight the..." → GROUNDING
        # "Use optical and SAR..." → SAR_OPTICAL_FUSION
        pass
```

PART 3: DATASETS DEEP DIVE

3.1 Dataset Analysis

text

DATASET BREAKDOWN	
BigEarthNet	PRIMARY TRAINING DATASET

	• Sentinel-1 SAR + Sentinel-2 Optical • 590,326 image patches • 43 land-use/cover classes • USE FOR: RS adaptation/fine-tuning • CRITICAL: Must use this for adaptation!
VRSBench	EVALUATION: Single image tasks • Captioning, grounding, VQA • High-resolution RS images • USE FOR: Testing single-image pipeline
RSVQA	EVALUATION: RS Visual QA • Presence, count, comparison questions • Rural + Urban splits • USE FOR: VQA evaluation
CDVQA	EVALUATION: Change Detection QA • Bi-temporal pairs + questions • USE FOR: Change analysis evaluation
ISRO/SAC Dataset	FINAL EVALUATION (Hidden) • Cartosat-2S optical + RISAT SAR • Pre-georeferenced, co-registered • Indian satellite data • CRITICAL: Your SAR-Optical fusion must work!

3.2 BigEarthNet Fine-tuning Strategy

What to Fine-tune:

1. Visual Encoder (CLIP/ViT):
 - Input: BigEarthNet S1+S2 patches
 - Task: Contrastive learning with text labels
 - Result: RS-adapted visual embeddings
2. Image-Text Alignment:
 - Use the text annotations in BigEarthNet
 - Align visual features with RS terminology
3. SAR Handling:
 - Separate encoder branch for SAR
 - Learn SAR-specific feature representations

Fine-tuning Pipeline:

BigEarthNet S2 Images + S1 SAR + Labels

↓
Data Preprocessing (normalize, patch)
↓
CLIP Fine-tuning with RS text pairs
↓
VQA Head Training on RSVQA
↓
Change Detection Head on CDVQA
↓

Evaluation on VRSBench

PART 4: WINNING STRATEGY

4.1 Differentiation Points

text

What AVERAGE teams will do:

- ✗ Use generic ChatGPT/LLaVA API without RS adaptation
- ✗ Simple pipeline without true agentic behavior
- ✗ No proper GeoTIFF handling
- ✗ Basic UI without visual evidence

- ✗ No confidence scoring
- ✗ No execution traces

What YOUR team should do to WIN:

- ✓ True RS-adapted models (BigEarthNet fine-tuning demonstrated)
- ✓ Genuine agentic orchestration with model registry
- ✓ Proper GDAL-based GeoTIFF processing
- ✓ Rich visual evidence (highlighted regions, change maps)
- ✓ Confidence scores with explanation
- ✓ Full execution trace/audit log
- ✓ ISRO Indian satellite data compatibility
- ✓ Clean, intuitive UI that impresses judges

4.2 Scoring Analysis
text

Evaluation Breakdown (Estimated):

Component	Weight	Our Strategy
Single-image VQA	High	RSVQA fine-tuned model
Captioning/Grounding	Medium	Implement BOTH
Change Analysis	High	ChangeFormer + CDVQA
SAR-Optical Fusion	High	Dual-encoder approach
RS Adaptation Proof	Critical	BigEarthNet demo
Agentic Orchestration	High	LangGraph agent
UI/UX Quality	Medium	React + Maps
ISRO Dataset Performance	Critical	Indian RS focus
Code Quality	Low-Med	Clean, documented
Presentation	Medium	Live demo ready

4.3 Implementation Priority Order
text

PHASE 1 - FOUNDATION (Week 1-2):

Priority: Get basic pipeline working end-to-end

1. Setup GeoTIFF processing with GDAL/Rasterio
2. Deploy pre-trained RemoteCLIP or GeoChat
3. Build simple FastAPI backend
4. Create basic React frontend
5. Implement agentic task classifier

PHASE 2 - CORE FEATURES (Week 3-4):

Priority: All mandatory features working

1. BigEarthNet fine-tuning (even partial shows adaptation)
2. VQA pipeline on RSVQA
3. Change detection with ChangeFormer
4. SAR processing pipeline
5. Agentic model routing

PHASE 3 - POLISH (Week 5-6):

Priority: Quality and winning features

1. Confidence scoring system
2. Visual evidence overlays
3. Execution audit traces
4. Report generation (PDF)
5. UI/UX improvements
6. ISRO satellite format support

PHASE 4 - EVALUATION PREP (Week 7-8):

Priority: Demo and presentation

1. Benchmark testing on all datasets
2. Performance optimization
3. Demo video/presentation
4. Edge case handling
5. Docker deployment

PART 5: DETAILED COMPONENT DESIGN

5.1 Agentic Controller Design

Python

Model Registry Design

```
MODEL_REGISTRY = {
    "task_classifier": {
        "model": "mistral-7b-instruct",
        "purpose": "Parse query intent",
        "input": "text query + image count + modality info"
    },
    "rs_vqa": {
        "model": "llava-1.6-finetuned-rsvqa",
        "purpose": "Answer questions about single RS images",
        "input": "single image + text query",
        "datasets": ["RSVQA", "VRSBench"]
    },
    "rs_captioning": {
        "model": "geochat-finetuned",
        "purpose": "Generate RS scene descriptions",
        "input": "single image",
        "datasets": ["VRSBench"]
    },
    "rs_grounding": {
        "model": "grounding-dino-rs-adapted",
        "purpose": "Locate regions from text description",
        "input": "single image + text query",
        "output": "bounding boxes + masks"
    },
    "change_detection": {
        "model": "changeformer",
        "purpose": "Detect changes between temporal pairs",
        "input": "image pair (t1, t2)",
        "output": "change map + change description"
    },
    "change_vqa": {
        "model": "cdvqa-model",
        "purpose": "Answer change-related questions",
        "input": "image pair + query",
        "datasets": ["CDVQA"]
    },
    "sar_optical_fusion": {
        "model": "dual-encoder-fusion",
        "purpose": "Joint SAR+Optical analysis",
        "input": "optical image + SAR image",
        "output": "fused analysis + complementary insights"
    },
    "rs_adapter": {
        "model": "bigearthnet-clip",
        "purpose": "RS-adapted visual encoding",
        "training": "BigEarthNet S1+S2",
        "base": "CLIP-ViT-B/32"
    }
}
```

Task Classification Logic

```
TASK_ROUTING = {
    "describe|caption|what is|explain|overview": "CAPTIONING",
    "highlight|locate|where is|show me|find": "GROUNDING",
```

```

    "changed|difference|before after|temporal": "CHANGE_DETECTION",
    "sar|radar|optical together|combined": "SAR_OPTICAL_FUSION",
    "how many|count|is there|does it have": "VQA",
    "default": "VQA"
}

```

5.2 Input Validation System

Python

```
class InputValidator:
```

```

    def validate(self, images, query):
        results = {
            "valid": True,
            "errors": [],
            "warnings": [],
            "image_config": None
        }

        # Check formats
        for img in images:
            if not self.is_supported_format(img):
                results["errors"].append(f"Unsupported format: {img.format}")

        # Check modality
        modalities = [self.detect_modality(img) for img in images]
        # → "optical" | "sar" | "multispectral"

        # Determine configuration
        if len(images) == 1:
            results["image_config"] = "SINGLE"
        elif len(images) == 2:
            if self.is_cross_modal_pair(modalities):
                results["image_config"] = "CROSS_MODAL"
            elif self.is_temporal_pair(images):
                results["image_config"] = "BITEMPORAL"
            else:
                results["warnings"].append("Cannot determine pair type")

        # Check co-registration for pairs
        if len(images) == 2:
            if not self.check_coregistration(images):
                results["warnings"].append(
                    "Images may not be co-registered - results may be
inaccurate"
                )

        return results

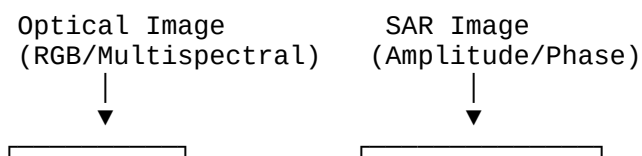
    def detect_modality(self, image):
        # Check band count, metadata
        # SAR: typically 1-2 bands, specific metadata
        # Optical: 3+ bands
        # Multispectral: many bands
        pass

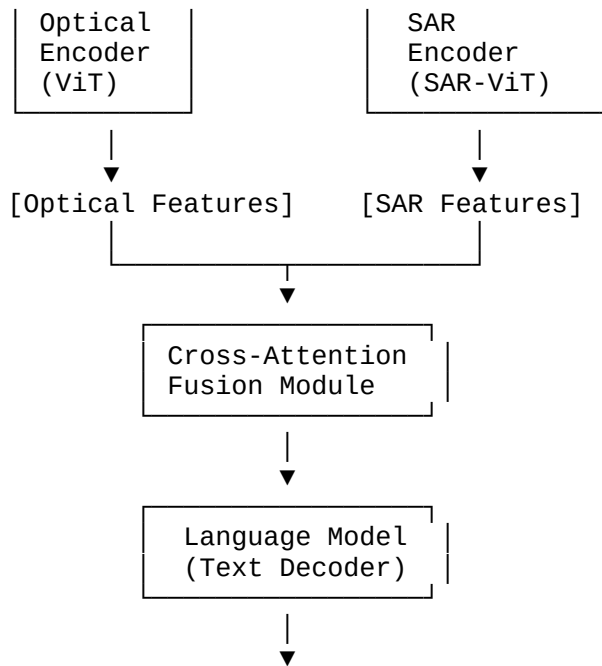
```

5.3 SAR-Optical Fusion Architecture

text

SAR-Optical Fusion Pipeline:



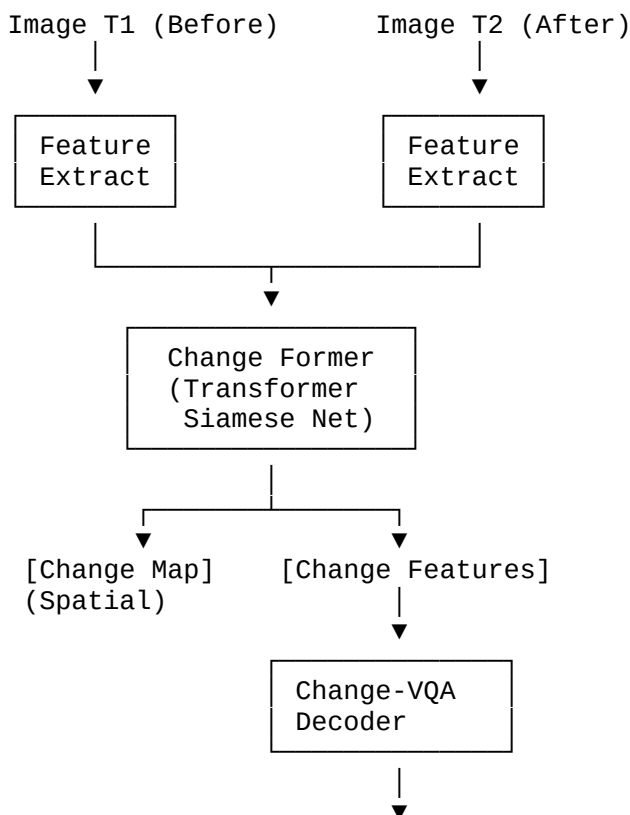


"The optical image shows dense urban area. SAR backscatter confirms high-density built-up structures. Water body detected at NW corner (confirmed by both sensors)"

5.4 Change Detection Pipeline

text

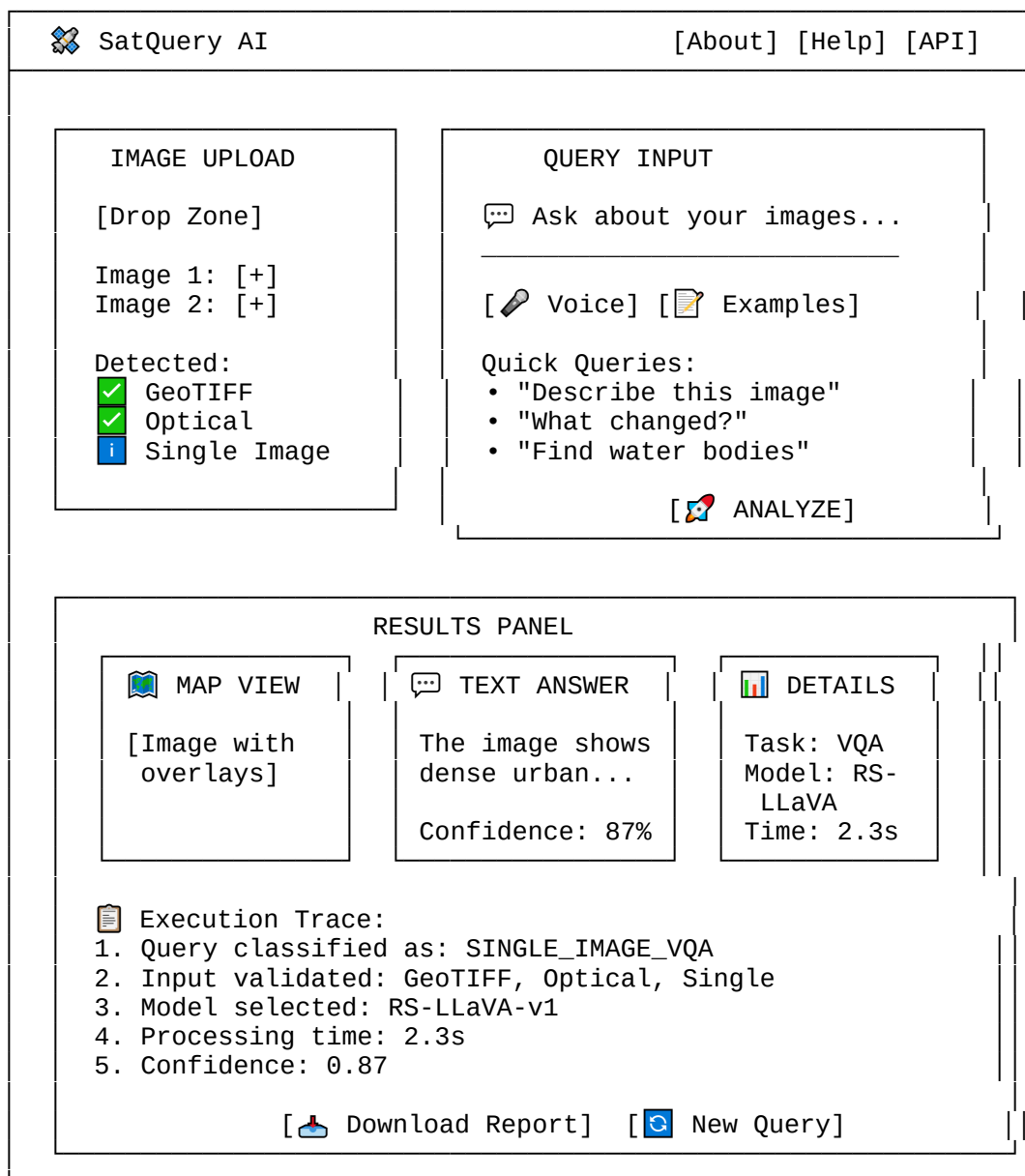
Bi-temporal Change Analysis:



"Between 2020-2023, significant urban expansion detected in NE sector (red regions on map). Approximately 15% increase in built-up area. Forest cover reduced by ~8%."

6.1 Frontend Layout

text



6.2 Key UI Features to Implement

text

Must-Have UI Features:

1. Drag-and-drop image upload
2. Real-time format validation
3. Image preview with metadata display
4. Natural language query input
5. Loading animation during processing
6. Side-by-side image comparison (for pairs)
7. Interactive change map overlay
8. Confidence meter/visualization
9. Execution trace panel (collapsible)
10. PDF report download
11. Query history
12. Example queries for each task type

PART 7: POTENTIAL CHALLENGES & SOLUTIONS

7.1 Technical Challenges

text

CHALLENGE 1: Computing Resources

Problem: Fine-tuning large models requires GPUs

Solution:

- Use Google Colab Pro / Kaggle for fine-tuning
- Use quantized models (4-bit) for inference
- LoRA/QLoRA for efficient fine-tuning
- Use pre-existing RS-adapted models (RemoteCLIP, GeoChat)

CHALLENGE 2: BigEarthNet Fine-tuning Time

Problem: Full dataset is massive

Solution:

- Use subset (10-20% of data)
- Show before/after metrics to prove adaptation
- Use transfer learning from existing CLIP
- Document the fine-tuning approach clearly

CHALLENGE 3: SAR Processing Complexity

Problem: SAR images need special preprocessing

Solution:

- Use SNAP (ESA) for SAR preprocessing
- Normalize to dB scale
- Use existing SAR processing libraries
- Focus on amplitude images initially

CHALLENGE 4: Co-registration Verification

Problem: How to verify if images are co-registered

Solution:

- Check metadata bounding boxes
- Overlap calculation
- Visual alignment check
- Warn user but proceed with caveat

CHALLENGE 5: ISRO/Indian Satellite Data

Problem: Cartosat-2S and RISAT have unique formats

Solution:

- Research Cartosat-2S image specifications
- Test with similar resolution data
- Ensure GeoTIFF pipeline is robust
- RISAT-1 SAR data is similar to Sentinel-1

7.2 Pre-trained Models You Can Use (No Training Needed)
text

IMMEDIATELY AVAILABLE RS-ADAPTED MODELS:

1. RemoteCLIP

- └ Already fine-tuned on RS data
- └ GitHub: [ChenDeLong1999/RemoteCLIP](#)
- └ Use as visual encoder backbone

2. GeoChat

- └ RS-specific chat model
- └ Based on LLaVA
- └ Supports RS VQA and captioning

3. RSGPT

- └ Remote Sensing GPT
- └ VQA and captioning capabilities

4. ChangeFormer

- └─ SOTA change detection
- └─ GitHub: wgcbn/ChangeFormer
- └─ Works on bi-temporal pairs

5. Grounding DINO

- └─ Text-guided object detection
- └─ Can be adapted for RS grounding
- └─ Works on various image types

6. SEGMENT ANYTHING MODEL (SAM)

- └─ Can segment any region
- └─ Useful for grounding tasks
- └─ Combine with CLIP for RS

KEY INSIGHT: Use these pre-trained models as specialist tools in your agent! This satisfies "RS adaptation" if you fine-tune even one of them.

PART 8: FINE-TUNING PLAN (CRITICAL FOR WINNING)

8.1 Minimum Viable Fine-tuning text

You MUST demonstrate RS adaptation. Here's the minimum:

Option A: CLIP Fine-tuning on BigEarthNet (Recommended)

Dataset: BigEarthNet Sentinel-2 + Text Labels

Base Model: CLIP ViT-B/32

Method: Contrastive fine-tuning

Duration: 5-10 epochs on subset

Hardware: 1 GPU (RTX 3090 or Colab A100)

Training Pairs:

Image: Sentinel-2 patch

Text: "Agricultural land with multiple crop types"
(from BigEarthNet labels)

Expected Improvement:

Before: Generic CLIP → poor RS understanding

After: RS-CLIP → better land-cover alignment

SHOW THIS IMPROVEMENT IN YOUR DEMO!

Option B: LLaVA Fine-tuning on RSVQA

Base: LLaVA-1.6-7B

Data: RSVQA dataset

Method: LoRA fine-tuning

Result: RS-specific VQA model

Option C: Use existing + prove domain shift

- Load RemoteCLIP (already RS-adapted)
- Show it performs better than vanilla CLIP
- Call it "RS-adapted component" in your system
- This IS valid RS adaptation!

8.2 BigEarthNet Data Preparation

Python

```
# BigEarthNet Preprocessing Pipeline
```

```
class BigEarthNetPreprocessor:
```

```
    def prepare_training_data(self):
        # Load Sentinel-2 patches
```

```

s2_patches = self.load_s2_images() # 12 spectral bands

# Load Sentinel-1 patches
s1_patches = self.load_s1_images() # VV, VH polarizations

# Generate text descriptions from labels
labels = self.load_labels()
text_descriptions = self.generate_text(labels)

# Create image-text pairs for CLIP fine-tuning
training_pairs = [
    {
        "image": s2_patches[i],
        "text": text_descriptions[i],
        "sar": s1_patches[i]
    }
    for i in range(len(s2_patches))
]

return training_pairs

def generate_text(self, labels):
    # Convert label list to natural language
    # ["Continuous urban fabric"] → "Dense urban area with continuous built-
up fabric"
    templates = {
        "Continuous urban fabric":
            "Dense urban area with continuous residential buildings",
        "Arable land":
            "Open agricultural land used for crop cultivation",
        "Water bodies":
            "Natural or artificial water body including lakes and rivers",
        # ... all 43 classes
    }
    return [templates.get(l, l) for l in labels]

```

PART 9: PRESENTATION & DEMO STRATEGY

9.1 Demo Flow for SIH Presentation

text

WINNING DEMO SEQUENCE (15-20 minutes):

[2 min] Introduction

- Problem: "Non-experts can't extract insights from satellite data"
- Solution: "SatQuery AI - ask in plain English, get expert answers"
- Architecture overview slide

[3 min] Single Image Demo

- Upload GeoTIFF (Sentinel-2 image)
- Query: "Describe the land cover in this image"
- Show: Agent routing → RS-Captioning model → Results
- Query: "Is there any water body visible?"
- Show: VQA result with confidence score
- Query: "Highlight the forested areas"
- Show: Grounded result with visual overlay

[3 min] Change Detection Demo

- Upload bi-temporal pair (same location, different years)
- Query: "What changed between these two images?"
- Show: Agent identifying it as change detection task
- Show: ChangeFormer processing
- Show: Change map overlay + text description
- Query: "Has urban area increased?"
- Show: Change VQA answer with evidence

[3 min] SAR + Optical Demo

- Upload Sentinel-2 + Sentinel-1 SAR pair
- Query: "Use both images to identify water bodies and built-up areas"
- Show: Agent routing to SAR-Optical fusion
- Show: Dual encoder processing
- Show: Complementary analysis combining both sensors

[2 min] Agentic Orchestration Highlight

- Show execution trace panel
- "The system automatically selected these models..."
- Highlight: Model names, parameters, confidence, timing

[2 min] ISRO Compatibility

- Mention Cartosat-2S and RISAT support
- Show GeoTIFF metadata handling
- Demonstrate format validation

[2 min] Differentiation vs Generic AI

- Show generic ChatGPT failing on RS questions
- Show SatQuery AI giving correct, grounded answers
- Emphasize RS adaptation with BigEarthNet metrics

[2 min] Q&A Prep

9.2 What Judges Will Look For
text

Judge Checklist (Based on Problem Statement):

- ✅ "Is there RS-specific fine-tuning/adaptation?"
→ Show BigEarthNet training metrics
- ✅ "Does it handle GeoTIFF properly?"
→ Demo with actual GeoTIFF files
- ✅ "Is the orchestration truly agentic?"
→ Show model selection changing for different queries
- ✅ "Does change detection work?"
→ Show visual change map + description
- ✅ "Does SAR-Optical fusion add value?"
→ Show how SAR reveals what optical misses
- ✅ "Are there confidence scores?"
→ Show in UI prominently
- ✅ "Is there an execution trace?"
→ Show audit log panel
- ✅ "Can it handle ISRO satellite data?"
→ Mention Cartosat/RISAT compatibility specifically

PART 10: TEAM STRUCTURE & TASK DIVISION

10.1 Recommended Team Roles (6 members)

text

MEMBER 1: ML Lead / RS Adaptation

Tasks:

- BigEarthNet data preparation
- CLIP/VLM fine-tuning
- Model evaluation on benchmarks
- VQA model integration

MEMBER 2: Computer Vision Engineer

-
- Tasks:
- Change detection pipeline
 - ChangeFormer integration
 - SAR preprocessing
 - Visual output generation (change maps)

MEMBER 3: Agentic Systems Developer

- Tasks:
- Agent controller design
 - Task classification logic
 - Model registry implementation
 - Workflow orchestration

MEMBER 4: Backend Developer

- Tasks:
- FastAPI backend
 - GeoTIFF/GDAL processing
 - API endpoints
 - Docker containerization
 - Database setup

MEMBER 5: Frontend Developer

- Tasks:
- React.js UI
 - Map visualization (Leaflet)
 - File upload handling
 - Results display
 - Report generation

MEMBER 6: Research & Integration

- Tasks:
- Dataset research and curation
 - SAR-Optical fusion model
 - Integration testing
 - Documentation
 - Presentation preparation

PART 11: RISK ASSESSMENT

11.1 Risk Matrix

text

Risk	Impact	Likelihood	Mitigation
GPU unavailable for fine-tuning	HIGH	MEDIUM	Use Colab/Kaggle Plan B: RemoteCLIP
ISRO dataset format incompatibility	HIGH	HIGH	Generic GeoTIFF + research specs
VQA accuracy too low on benchmarks	HIGH	MEDIUM	Multiple models, ensemble approach
SAR processing complexity	MEDIUM	HIGH	Start early, use SNAP toolbox
Agent routing failures	MEDIUM	> MEDIUM	Rule-based fallback + LLM routing
Time management	HIGH	HIGH	Phase-based plan

PART 12: QUICK-WIN STRATEGIES

12.1 Things That Will Impress Judges Immediately
text

🏆 QUICK WINS (High Impact, Lower Effort):

1. Use RemoteCLIP directly
 - Immediately have RS-adapted visual encoder
 - Cite it as domain-adapted component
 - 2-3 days to integrate
2. GeoChat Integration
 - Already fine-tuned VLM for RS
 - Gives captioning + VQA instantly
 - 3-4 days to integrate
3. Beautiful Execution Trace UI
 - Judges love seeing the "thinking"
 - Just log agent decisions to UI
 - Shows understanding of agentic systems
4. Change Map Visualization
 - ChangeFormer gives binary change maps
 - Overlay in red/green on original image
 - Very visual and impressive for judges
5. Confidence Meter Widget
 - Simple progress bar showing confidence
 - "High confidence (87%)" looks professional
 - Easy to implement, big visual impact
6. "Why this model was selected" explanation
 - Show agent reasoning: "Detected bi-temporal pair"
 - Selected change detection workflow"
 - Judges see agentic behavior clearly

12.2 Minimum Viable Product (If Short on Time)
text

MVP that still satisfies ALL mandatory requirements:

Week 1-2: Core Pipeline

- └ FastAPI + React setup
- └ GeoTIFF loading with GDAL
- └ RemoteCLIP visual encoder (RS-adapted ✅)
- └ LLaVA-based VQA (satisfies VQA ✅)

Week 3-4: Essential Features

- └ GeoChat for captioning (satisfies captioning ✅)
- └ ChangeFormer change detection (satisfies change ✅)
- └ Dual encoder for SAR-optical (satisfies cross-modal ✅)
- └ Simple rule-based agent routing (satisfies agentic ✅)

Week 5-6: Polish

- └ Confidence scores
- └ Execution trace UI
- └ Demo preparation

This MVP would score reasonably well!

The extras (grounding, reports, etc.) are bonus points.

FINAL RECOMMENDATIONS

🎯 Top 10 Actions to Take Right Now

text

1. IMMEDIATELY: Download BigEarthNet dataset (start now, it's large)
 2. THIS WEEK: Set up RemoteCLIP + GeoChat in your environment
→ These give you RS adaptation immediately
 3. THIS WEEK: Set up GDAL/Rasterio for GeoTIFF processing
→ This is the foundation of everything
 4. WEEK 1: Design your agent architecture on paper
→ Model registry, routing logic, workflow
 5. WEEK 1: Start ChangeFormer setup for change detection
→ This is the most complex pipeline, needs most time
 6. WEEK 1-2: Begin BigEarthNet fine-tuning
→ Even partial results show RS adaptation
 7. WEEK 2: Build basic FastAPI backend with model endpoints
 8. WEEK 2: Start React frontend with upload + results display
 9. WEEK 3: Integration testing with all benchmarks (RSVQA, CDVQA, VRSBench)
 10. WEEK 4+: Polish UI, add confidence scores, execution traces
- 🏆 Winning Formula
text

WINNING = RS Adaptation (BigEarthNet/RemoteCLIP)
 + True Agentic Routing (not fake pipeline)
 + Working Change Detection (visual map)
 + SAR-Optical Fusion (even basic)
 + Beautiful UI (visual evidence)
 + ISRO Compatibility Emphasis
 + Clean Demo (prepared, no errors)
 + Strong Presentation (explain the WHY)

Summary Assessment

Aspect	Difficulty	Your Chance
Technical Complexity	Very High	Win if executed well
Pre-existing Tools	Many available	Advantage
Novelty Required	Medium	Agentic framework is key
Winning Probability	High	If team is strong

This is a winnable problem if you have a strong ML + backend + frontend team and start immediately. The key differentiator is the agentic orchestration + RS adaptation proof - most teams will submit generic VLM wrappers without true domain adaptation.

Your biggest advantage: Understanding that ISRO wants to see Cartosat-2S/RISAT compatibility means you should specifically test and mention Indian satellite support throughout your presentation.

BEFORE SatQuery AI:

```

Expert uploads satellite image
↓
Opens complex GIS software
↓
Runs multiple specialized programs
↓
Waits hours for analysis
↓

```

Gets technical output

↓

Translates to simple answer

↓

Total time: Days

AFTER SatQuery AI:

Anyone uploads satellite image

↓

Types simple question in English

↓

Gets visual answer with explanation

↓

Total time: 30 seconds